MINI PROJECT 1 – COMP370: SOFTWARE ENGINEERING

INTRODUCTION

The goal of this project is to design and simulate a server redundancy management system (SRMS) that ensures high availability of services by using a primary backup server model. In this system, one server actively handles client requests while other servers remain on standby and automatically take over if the primary server fails.

STEP 1: Understanding the Problem and System Requirements

This section explains what the system is expected to do and the constraints under which it must operate.

Who is involved in this system?

The main stakeholders for this system include:

- System Administrators, who monitor servers and manage failures
- End Users (Clients), who send requests and expect uninterrupted service
- Developers and Maintainers, who build and improve the system

- Instructors and Evaluators, who assess the quality of the design and implementation

What problem are we solving?

The system is designed to simulate a cluster of servers where one server acts as a primary server and other acts as backups. Client requests are handled by a primary server, and when that server becomes unavailable, a backup server automatically takes over without interrupting service.

What the system must be able to do (Functional Requirements)

The SRMS must:

- Allow clients to send requests and receive responses from the active primary server.
- Ensure that only one server acts as the primary at any time
- Monitor server health using heartbeat messages
- Automatically promote a backup server when the primary fails
- Allow clients to discover which server is currently the primary
- Record major system events such as server failures and failovers
- Support simulated failures for testing purposes

How well the system should perform (Non-Functional Requirements)

The system should also meet the following quality expectations:

- Detect primary server failure with a small number of missed heartbeats
- Resume service quickly after failover
- Avoid situations where more than one primary server exists
- Produce clear and timestamped logs
- Scale to multiple servers without major redesign

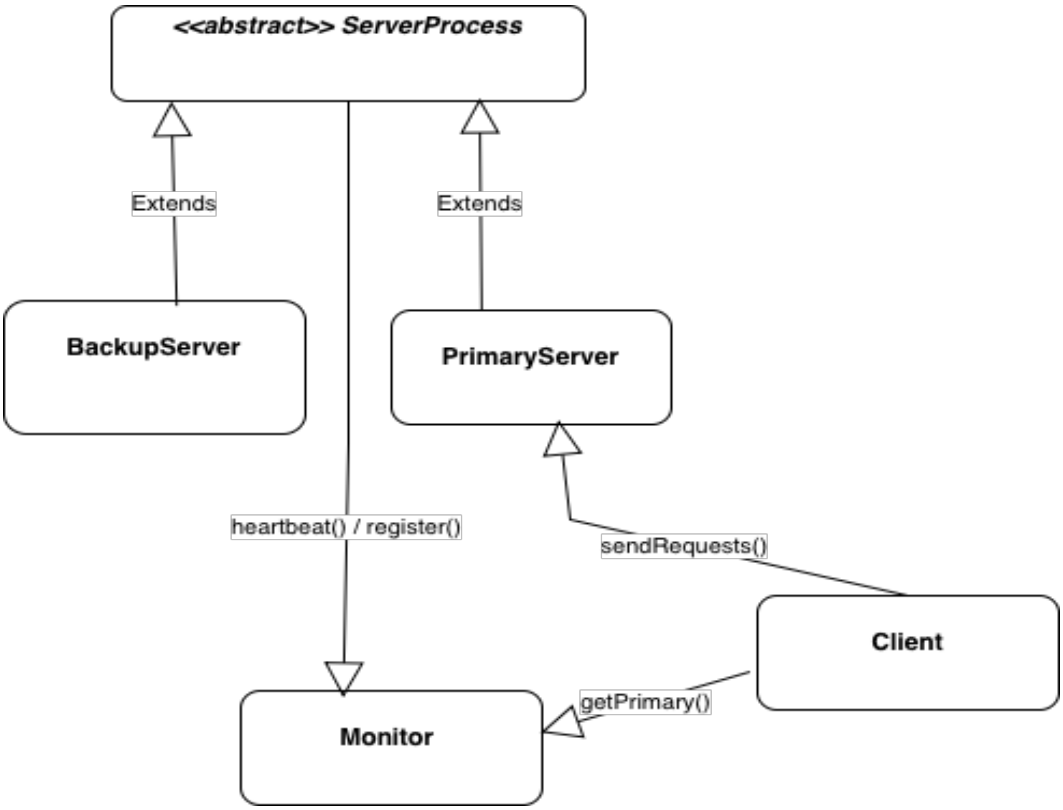
STEP 2: SYSTEM DESIGN USING UML

This section describes how the system is structured and how different components interact using UML diagrams.

Overview of the design

To better understand and communicate how the SRMS works, UML diagrams are used to represent both the structure and the behaviour of the system. These diagrams help ensure that the design is clear before coding begins.

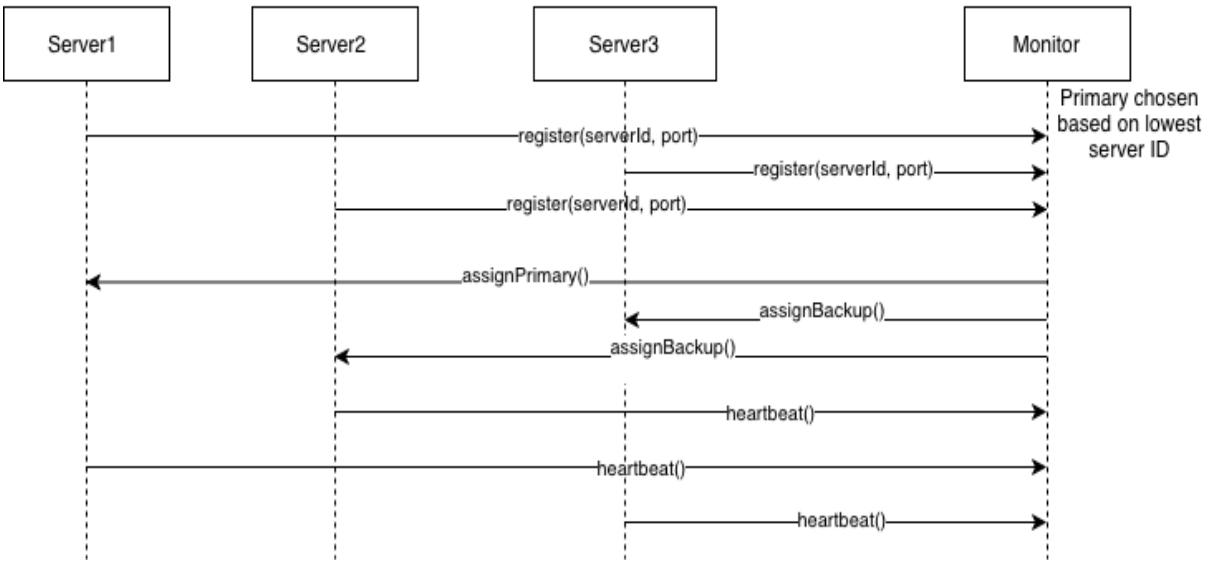
2.1 UML CLASS DIAGRAM



The class diagram shows the structure of SRMS system. ServerProcess is an abstract class that represents common behaviour of all servers. PrimaryServer and BackupServer extends this class. The Monitor tracks server health and manages failover. The Client interacts with the system by querying the monitor and sending requests to the primary server.

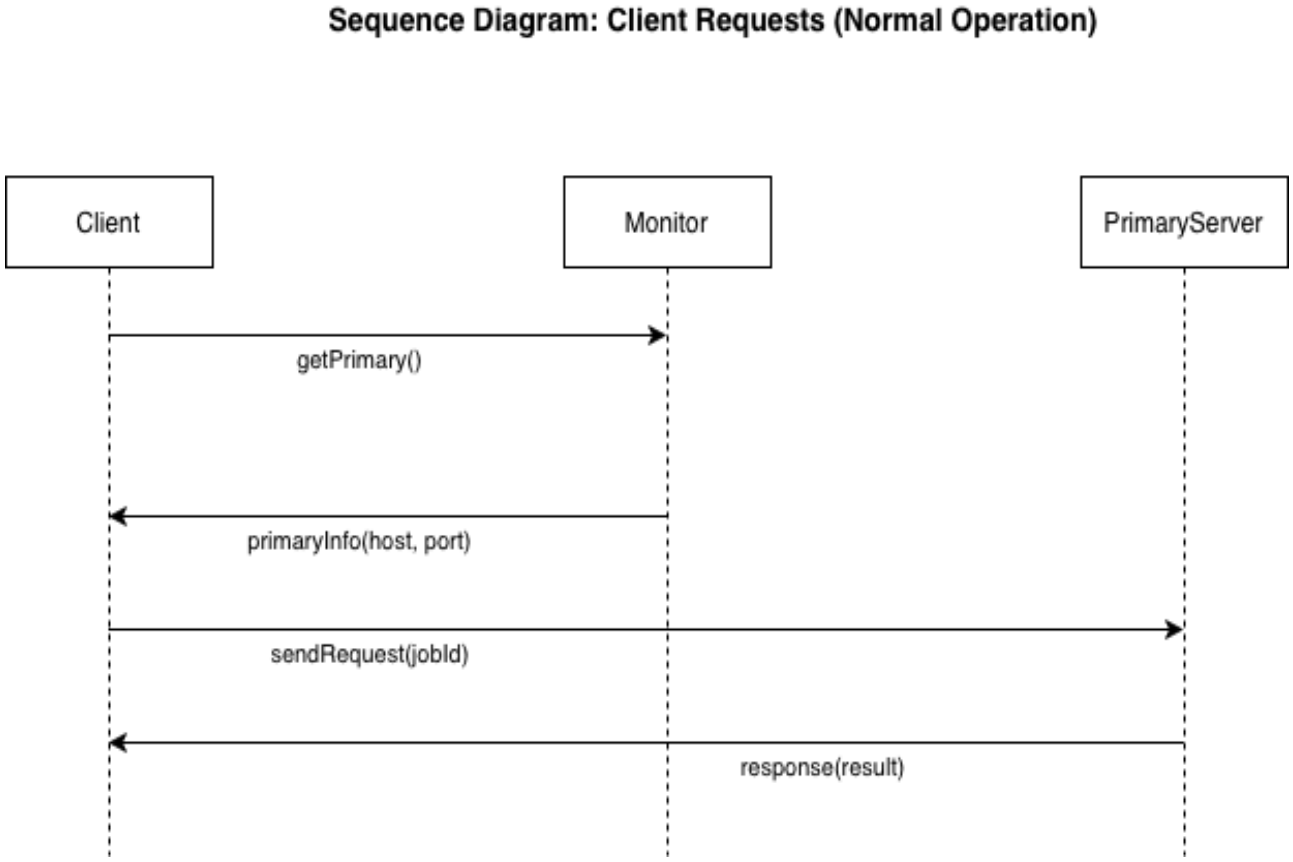
2.2 SEQUENCE DIAGRAM: Startup and Leader Election

Sequence Diagram : Startup and Leader Election



This sequence diagrams illustrates how server register with the monitor during startup. The monitor assigns one server as the primary and the other as backups. After role assignment, server begins sending heartbeat messages so the monitor can track their health.

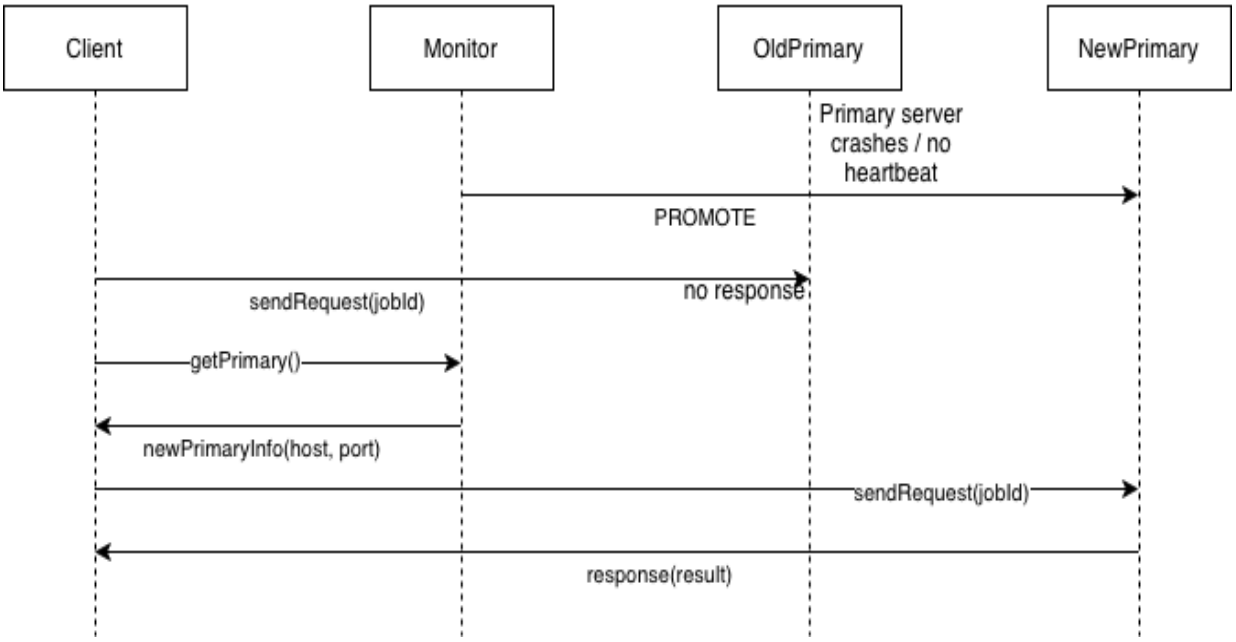
2.3 SEQUENCE DIAGRAM: Client Requests (Normal Operations)



This diagram shows how a client sends a request during normal system operations. The client first contracts the monitor to discover the primary server and then sends its request to the primary server, which processes the request and returns a response.

2.4 SEQUENCE DIAGRAM: Client Request During Failover

Sequence Diagram: Client Request During Failover



This diagram demonstrates how the system behaves when the primary server fails. The monitor detects the failure and promotes a backup server to become the new primary. The client then retires its request using the new primary server, ensuring continuous service.

These UML diagrams provide a clear and structured view of the system before implementation. They ensure that all components and interactions are well understood and serve as a strong foundation for building the SRMS.